



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

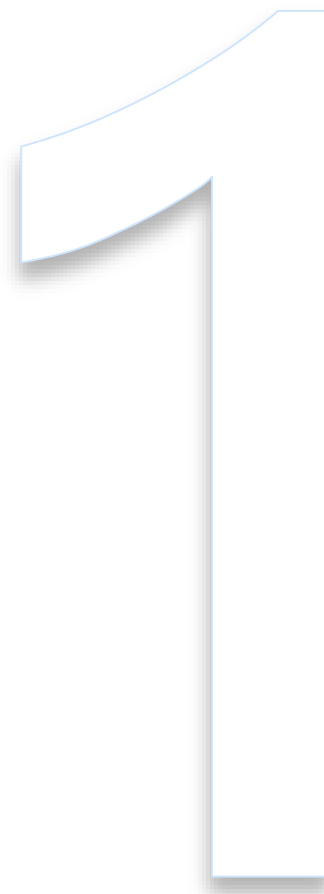
Функционал программчлалын парадигм

Лекц 10

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

- Төлөвгүй тооцоолол
 - Илэрхийлэл ба функц
 - Бууралт тооцоолол
 - Үндсэн бүрдэл
- Тооцоолох/үнэлэх (Evaluation)
 - Утга, Стратеги
- Функционал PL
 - Pattern matching
 - Функционал стайлын программчлал
 - Төрөл, Императив тал
- Дүгнэлт



Функционал программчлал (FP)-ын парадигм:

Тооцоолол нь төлвийн өөрчлөлт дээр биш илэрхийллийг хувиргах (rewriting) зарчмаар явагддаг.

*Уг парадигмын "дагнасан (pure)" хэлбэрийн хэл нь **санах ойн ойлголт ашигладаггүй** (ямар ч **гадаад нөлөө (side effect)** байхгүй).*

Орчныг байгуулснаас хойш илэрхийлэл үргэлж ижил утгатай байна.

Төлөвгүй тооцоолол

- *Уламжлалт тооцоолол: Төлөв өөрчлөлт дээр суурилдаг (Императив)*
 - *Modifiable хувьсагч*: тооцооллын явцад өөр өөр утга оноох боломжтой
 - *Утга олголт*: Нэрийн холболтыг нь өөрчлөлгүй байршилд нь утгыг засах (1940s, Von Neumann Machine, Анхны electronic stored-program computers)
- *Функционал программчлалын парадигм: (high-order) функц, рекурс*

Онолын хувьд тооцоолол нь илэрхийлэлийн дахин бичилтээр (rewriting) буюу зөвхөн орчны өөрчлөлтөөр явагддаг (санах ойн ойлголтоос ангид).

- Төлөв, modifiable хувьсагч, утга олголт байхгүй
- *Төлөвгүй (stateless) тооцооллын загвар*: Хяналтын дарааллын хувьд (төгсгөлгүй) давталт “алга болж”, рекурс үлддэг. (Төгсгөлөг давталтыг өндөр эрэмбийн функцээр илэрхийлнэ)
- 1930s, *λ -calculus* (тооцоологдом функцийн загвар):
 - *Императив нэмэлттэй*: List, Scheme, ML (сургалтанд тохиромжтой), Erlang
 - *Цэвэр функционал*: Miranda, Haskell

STATELESS ТООЦООЛОЛ >> Илэрхийлэл, функц

- Ердийн математик практикт: $f(x) = x^2$; хэрэв $x = 2$ өгвөл $f(x) = 4$ болно.
 - $f(x)$ нь 1.) f гэсэн нэрийг танилцуулах; 2.) үр дүнг илэрхийлэх;
 - " $f(x)$ " *нэр* нь f -ийг x *формал* параметртэй тодорхойлох бөгөөд аргумент дээр f хэрэгжсэн байх *хувиргалтыг (их бие)* төлөөлнө.
- Функцийн нэр ба "их бие"-ийг зохиомол хэлэнд ялгах ёстой (ML синтакс)

```
val f = fn x => x * x;
```

- f нэр нь x -г $x * x$ болгон хувиргах, fn нь функц гэдгийг тэмдэглэх.
 - Функц нь *илэрхийлэх (expressible) боломжтой утга* (комплекс илэрхийллийн тооцооллын үнэлгээний үр дүн байх) байна
- Функцийн аргументын синтакс $f(2)$; $(f\ 2)$; болон $f\ 2$:

```
val four = f 2;
```

- Функцэд заавал нэр өгөхгүйгээр **бичих, ажиллуулах** боломжтой.

```
(fn y => y+1) (6);
```

- Хэрэглээг шууд зэрэгцүүлэн (хаалтгүй) тэмдэглэж болох ба зүүн талруугаа холбоостой гэж үзнэ (prefix).

```
(... (g a1) a2) ... ak) -гийг g a1 a2 ... ak
```

- Нэг функционал илэрхийлэл дотор нөгөөг хориглохгүй (нэргүй функц)

```
val add = fn x => (fn y => y+x);
```

- Дараах байдлаар дуудаж хэрэглэх боломжтой:

```
val three = add 1 2;  
val addtwo = add 2;  
val five = addtwo 3;
```

- `val`, `fn` тэмдэглэгээ нь чухал боловч бага зэрэг нуршуу.

```
val f = fn x => x*x; -гийг
```

```
fun f x = x*x;
```

- Ерөнхий тохиолдолд

```
val F = fn x1 => (fn x2 => ... (fn xn => body) ...);
```

- Зүгээр л *syntactic sugar* (товч бичихэд тохиромжтой)

```
fun F x1 x2 ... xn = body;
```

- Нөхцөл илэрхийлэлтэй рекурсив тодорхойлолт: Жнь: factorial

```
fun fatt n = if n=0 then 1 else n*fatt (n-1);
```

Бууралт (Reduction): Хэрэв арифметик функц болон нөхцөлт илэрхийллийг тооцохгүй бол комплекс илэрхийллийг утга болгон хувиргах(үнэлгээ) процедурыг **rewriting** процесс гэж тодорхойлж болно.

- Комплекс илэрхийлэлд "аргументад хэрэглэгдэх функц" хэлбэрийн дэд илэрхийлэл нь
 - Эхлээд формал параметрт байгаа функцийн их биеээр текстийн хувьд,
 - Дараа нь өөрийн ээлжин дээр бодит параметрээр солигддог.
- Дараах тохиолдолд тооцоолол сарнисан (diverges) гэх бөгөөд үр дүн нь тодорхойгүй болсон гэж хэлнэ. r-ийн үнэлгээг агуулсан тооцоолол бүр дахин бичилт

```
fun r x = r(r(x));
```



```
fact 3 → (fn n => if n=0 then 1 else n*fact(n-1)) 3
      → if 3=0 then 1 else 3*fact(3-1)
      → 3*fact(3-1)
      → 3*fact(2)
      → 3*((fn n => if n=0 then 1 else n*fact(n-1)) 2)
      → 3*(if 2=0 then 1 else 2*fact(2-1))
      → 3*(2*fact(2-1))
      → 3*(2*fact(1))
      → 3*(2*((fn n => if n=0 then 1 else n*fact(n-1)) 1))
      → 3*(2*(if 1=0 then 1 else 1*fact(1-1)))
      → 3*(2*(1*fact(0)))
      → 3*(2*(1*((fn n => if n=0 then 1 else n*fact(n-1)) 0)))
      → 3*(2*(1*(if 0=0 then 1 else n*fact(n-1))))
      → 3*(2*(1*1))
      → 6
```

Синтаксийн хувьд цэвэр FL нь зөвхөн илэрхийлэлтэй (командгүй)

- Data, нөхцөлт илэрхийлэл дээрх **боломжит утгууд** болон **анхдагч үйлдлүүдээс** гадна илэрхийллийг тодорхойлох хоёр үндсэн бүтэц байдаг:
 - exp илэрхийлэл, x параметр дээрх **хийсвэрлэл**: x -ийг exp болгон хувиргах тэмдэглэгээг ($\text{fn } x \Rightarrow \text{exp}$) зөвшөөрдөг.
 - f_exp илэрхийллийн өөр нэг a_exp илэрхийлэл дэх **хэрэглээ**: f_exp функцийн a_exp аргумент дээрх хэрэглээ ($f_exp\ a_exp$)
- Программ болон data хооронд бүрэн homogeneity оршдог.
 - Функцийг нөгөөд дамжуулах, (high-order) функцийн үр дүнгээр буцаах

Семантик хувьд, программ нь утга тодорхойлолтын цуваа байна

- Тодорхойлолт бүр нь орчинд шинэ холбоос үүсгэнэ
- Дурын комплекс илэрхийллийн үнэлгээг шаарддаг
- Өндөр эрэмбийн болон рекурсив функц нь уян хатан, хүчтэй болгодог.

- Илэрхийллийг **утга шууд заасан хэлбэртэй** болтол **хоёр үйлдлийг** давтах (хялбар *симболик rewriting* буюу **бууралт(reduction)**-ийг ашигладаг):
 1. Тухайн орчин дахь энгийн хайлт: Identifier орчинд олдох үед түүнийг тодорхойлолтоор нь солино.
 2. Аргументад хэрэглэгдэх функционал илэрхийлэлтэй ажилладаг ба хуулах (сору) дүрмийн нэг хувилбарыг ашигладаг
 - **Redex:** Буурч болох илэрхийлэл: $((\text{fn } x \Rightarrow \text{body}) \text{ arg})$
 - **Reductum:** өмнөх редексийн body доторх x-ийн байршил бүрт arg-ийн хуулбарийг сольсон байх илэрхийлэл (хувьсагч capture хийхээс зайлсхийх).
 - **β -дүрэм:** Редекс дотор дэд илэрхийлэл болж гарч ирэх expr илэрхийлэл нь expr1 болж буурдаг (тэмдэглэгээ: $\text{expr} \rightarrow \text{expr1}$)
 - редексийг түүний редектумаар солин expr1-ийг expr-ээс гарган авдаг.

#> Үнэлгээ (Evaluation) - Тооцоолол

Дор хаяж хоёр асуудал нээлттэй байна:

- Бууралтын төгсөх нөхцөл нь юу вэ
 - "утга шууд заасан хэлбэр" гэж юу вэ?
- β -дүрэмд нарийвчилсан ямар семантик байх ёстой вэ,
 - Зөвхөн боломжит хувьсагчийг олж авах (capture) тухай биш,
 - Нэг илэрхийлэлд нэгээс олон редекс байх үед
 - Дахин бичих явцад дагаж мөрдөх шаардлага

- **Утга:** дахин бичигдэхгүй илэрхийлэл. FL-д Анхдагч төрлийн, **функцийн**

```
val G = fn x => ((fn y => y+1) 2);
```

- G-тэй ямар утга холбох нь тодорхойгүй бөгөөд 2 тохиолдолтой

```
fn x => 3
```

```
fn x => ((fn y => y+1) 2)
```

- FL байнга сүүлийнхийг ашигладаг – хийсвэрлэл "доор" үнэлгээ байхгүй.
 - `fn x => expr` хэлбэрийн илэрхийлэл бүр нь утга
 - аргументад хэрэглэх хүртэл `expr`-ийн редексийг хэзээ ч дахин бичихгүй.

EVALUATION ➤ Үнэлгээний стратеги

- FL нь high-order функцтэй тул *үнэлэх стратеги* бол илүү суурь асуудал.

```
fun K x y = x;  
fun r z = r(r(z));  
fun D u = if u=0 then 1 else u;  
fun succ v = v+1;  
  
val v = K (D (succ 0)) (r 2);
```

*Нэгээс олон редекстэй
илэрхийлэл.*

*Аль утгыг v-тэй холбох, энэ
нь хэрхэн тогтоогдох вэ?*

- β-дүрэм нь дангаараа тийм ч их ашиггүй: v-ийн баруунд 4 редекс
- Хамгийн зүүн захын дүрмээр ч 3 редекс шаталсан тул ашиггүй

```
K (D (succ 0))  
D (succ 0)  
succ 0  
r 2
```

```
K (D (succ 0))  
D (succ 0)  
succ 0
```

- Хамгийн зүүн захын эрэмбэ + *утгын/нэрийн/lazy* стратеги хэрэгтэй.

- **Утгаар үнэлэх:** applicative-order, or eager, or innermost
 - Аргументийн илэрхийлэл нь утга болсны дараа л редекс үнэлэгдэнэ.
 - Өмнөх жишээний хувьд β -дүрмээр r (r 2)-д хүрч, улмаар r (r (r 2)) нь сарних тул v -д холбох утга байхгүй
- **Нэрээр үнэлэх:** normal order or outer-most
 - Аргументийн хэсэг эхлээд буураагүй байхад редекс буурдаг.
 - Давхцал гарвал редексийг олон үнэлнэ. Энд ялгаатай утгыг олж авах боломжийг бий болгоно. Зардал их.
- **Lazy үнэлгээ:** Нэрээр үнэлэхтэй адил ч редекс "хуулбар"-тай анх таарахдаа утгыг нь хадгалдаг. Дахин таарвал энэ хуулбарыг ашиглана.
- Lisp, Scheme, ML, Erlang зэрэг нь утгын стратегийг (мөн тэд үндсэн императив талуудыг багтаасан учраас) ашигладаг
- Миранда, Хаскелл (цэвэр функциональ хэлүүд) нар lazy үнэлгээг ашигладаг.

#» Функционал хэл дээрх программчлал (Programming in FL)

- Бидний судалж буй механизмууд тооцоологдом функцийн программыг илэрхийлэхэд хангалттай (Тьюринг-гүйцэд хэлийг бүрдүүлдэг)
- FL бүрийн гол цөм нь хэдий ч үүнийг бодит программчлалын хэл болгон ашиглахад хэтэрхий хатуу.
- Тиймээс FL бүр энэ цөмийг өөр янзын олон механизмтай өргөн контекст дотор багтаасан байдаг (энэ нь программчлалыг хялбар, илүү илэрхийлэх чадвартай болгох зорилготой).

- Бидний ашиглаж буй орчны глобал тодорхойлолтын механизм нь орчин үеийн хэлний хувьд хэтэрхий жижиг бүтэц. Тиймээс эдгээрийг нэвтрүүлэх ил механизмуудыг өгөх нь зүйтэй, жишээлбэл:

```
let x = exp in exp1 end
```

- зөвхөн `exp1`-н хамрах хүрээнд `exp`-ийн утгад `x`-г холбох.
- Функционал илэрхийлэл нь бодит параметртэй холбогдсон функцийн формал параметрээс бүрдэх шаталсан хамрах хүрээ болон холбогдох орчныг шууд нэвтрүүлж байна. Үнэлгээний үүднээс авч үзвэл өмнөх бүтэц нь дараах бичиглэлийн syntactic sugar :

```
(fn x => exp1) exp.
```

- Тодорхойлолтыг орчинд анх удаа хадгалахад компайлдсан код үүсгэдэг үр ашигтай хэрэгжүүлэлтүүд байдаг ч энэ загвар нь интерпретатор дээр суурилсан хэрэгжилтийг шууд санал болгодог.

- Функционал хэл бүр нь интерактив орчинтой байдаг.
 - Хэл нь хийсвэр машин үнэлдэг илэрхийллүүдийг оруулан утгыг нь буцаана.
 - Тодорхойлолт нь глобал орчныг өөрчлөх хэсэгчилсэн илэрхийллүүд байна (мөн утга буцааж болно).
- Тодорхойлолтыг орчинд анх хадгалахад компайлдсан код үүсгэх үр ашигтай хэрэгжилтээс үл хамааран интерпретатор дээр суурилсан хэрэгжилтийг шууд санал болгодог.

(PROGRAMMING IN FL) >> Паттерн тааруулах (Pattern Matching)

- Рекурсив функционал программчлалд нэг ярвигтай нь "if"-ээр терминал тохиолдлыг хуваарилах. Жнь, Фибоначчийн цуваа (маш үр ашиггүй)

```
fun Fibo n = if n=0 then 1
              else if n=1 then 1
                   else Fibo (n-1) + Fibo (n-2) ;
```

- ML, Haskell зэрэг хэлний паттерн тааруулах (pattern matching) механизм.

```
fun Fibo 0 = 1
  | Fibo 1 = 1
  | Fibo n = Fibo (n-1) + Fibo (n-2) ;
```

- Формал параметр нь identifier биш паттерн (схем) болж байна
- Функц бодит параметр дээр хэрэгжихэд схемтэй харьцуулж эхний тохирох паттерний их биеийг сонгоно.

- Ялангуяа бүтэцлэгдсэн төрлүүд, жнь жагсаалтад уян хатан. Энд :: нь "cons"

- 1 ["one", "two", "three"]
- 2 "zero" :: ["one", "two", "three"]
- 3 ["zero", "one", "two", "three"]

- Паттернд буй нэр бодит параметрыг заах формал параметр болно.

```
fun length list = if list = nil then 0  
                  else 1 + length(tl list);
```

```
fun length nil = 0  
  | length e::rest = 1 + length(rest);
```

*Ердийнхөөс
илэрхий товч
тодорхой*

- Хувьсагч нэг паттернд хоёр удаа гарахгүй. Жнь, жагсаалтын эхний 2 элемент = бол true:

- 3-р нь х-ийг 2 удаа агуулсан, хориотой

- Паттернд буй утгууд нь хоорондоо хамаарах эсэхийг шалгадаггүй.

```
fun Eq nil = false  
  | Eq [e] = false  
  | Eq x::x::rest = true  
  | Eq x::y::rest = false;
```

(PROGRAMMING IN FL)» Функционал хэв маягийн программчлал

- *Функционал хэв маягаар* программчлах нь *хувиршиггүй (immutable)* өгөгдлийг тойрсон (жижиг) функцийн (том) олонлогоор хийгддэг.
 - Тусгай зориулалтын программд зориулан программчлалын ерөнхий схемийг high-order функц байхаар тодорхойлж болно.
 - Жнь: "map, filter", императив "linear scan of a sequence"-тай эквивалент.

```
res=map(f,list_of_data)
```

```
res=[]  
for e in list_of_data:  
    res.append(f(e))
```

- Маш чухал: Программ схемийг илүү ашиглан кодын модуларыг нэмэгдүүлдэг.
 - testing, verification, correctness тал дээр илүү хялбар, үр ашигтай.
- Цэвэр програмчлалын хэлний онцлог нь ямар ч side effect-гүй байх
 - Том программ бичихэд төлөвтэй тооцооллоос зайлсхийх.
 - Программын correctness-г хянахад хялбар болгодог
- Scala: Нэрийг зарлах: **var** (утга оноох), **val** (өөрчлөх боломжгүй холболт)

- Төрлийг динамик шалгадаг Scheme-с бусад нь төрлийн хатуу статик системтэй.
- Төрлийн систем pair, list, "records" зэрэг шинэ төрлийг тодорхойлох боломжтой. Жнь ML-д:

```
fun add_p (n1 , n2 ) = n1 + n2 ;
```

- add_p нь int хосыг шаарддаг, n1-г утгатай, n2-г хоосон бол ажиллахгүй.
- Төрлийн системийнх нь чухал хэсэг бол функц төрөл
 - Функц нь тэмдэглэгдэх, илэрхийлэгдэх утгууд (хэрэв хэл нь императив оролцоотой бол хадгалагдах боломжтой)

- Төрөлтэй FL-д төрөлжүүлэх боломжгүй илэрхийлэл. Жнь:

```
fun F f n = if n=0 then f(1) else f("pippo");
```

- Өөр нэг хориглогдсон илэрхийлэл (төрөлжүүлэлтийг зөрчсөн) өөртөө хэрэглэх (self-application) юм:

```
fun Delta x = x x;
```

- Оноолтын зүүн талд байгаа тул 'a -> 'b хэлбэрийн төрөлтэй байх ёстой. Мөн Функцийн аргумент болох (баруун талд) тул функцэд шаардагдах төрөлтэй байх ёстой ('a төрлийн).
- x нь нэгэн зэрэг 'a болон 'a -> 'b төрлийн байх ёстой тул эдгээр хоёр илэрхийллийг "нэгдмэл" байлгах арга байхгүй.
- Scheme зэрэг төрлийн хатуу системгүй хэл Delta функцийг бичиж болно.

```
(Delta Delta)
```

- (4 3) гэж хэрэглэвэл int 4-ийг int 3-д хэрэглэх болж биелэлтийн үед хийсвэр машин алдаа өгнө (зүүн гар талын хэсэг нь функц биш).

- Олон FL-д гадаад нөлөөгөөр өөрчлөгдөх төлвийг бүхий императив механизмуудыг агуулагддаг.
- ML-д бодитоор засварлагдах боломжтой хувьсагч (reference cell) байдаг.

```
ref v          val n = !I + 1;          val I = ref 4;  
val I = ref 4;  I := !I + 1;             val J = I;
```

- Огт өөр санах ойн хэмжээтэй утгуудыг reference cell-д хадгалах боломжтой. Далд явагдана. Жнь, жагсаалт ба тэмдэгт мөр.

```
val S = ref "pear";  
val L = ref ["one", "two", "three"];
```

- Өөрчлөгдөж боломжтой хувьсагчдаас гадна ML нь дараалсан бичлэг (;) болон loop гэх мэт императив хяналтын бүтцийг бий болгодог.

Дүгнэлт

- Тооцооллын эхлэн суралцагчид үр ашигтай программ зохиох, бичих нь хамгийн чухал гэж үздэг. ПХ-ийн инженерчлэлд судалгаа том зургаар хардаг.
 - ПХ-ийн төслийн зөв, уншигдах чадвар, хадгалах чадвар, найдвартай байдал.
 - *Эдийн засаг*: нийт зардлын тавиас дээш хувийг эзэлдэг;
 - *Нийгэм*: ПХ-ийн засвар үйлчилгээ (уншигдах чадвараас шалтгаална) хэдэн арван жилийн хугацаанд олон зуун хүнийг хамарч болно.
 - *Ёс суртахуун*: ПХ-ийн системийн найдвартай, зөв эсэхээс олон мянган хүний амь нас, эрүүл мэнд хамаарна.
- Гадаад нөлөө бүхий программын талаар дүгнэлт хийх нь хэцүү, үнэтэй байдаг. Эсрэгээр, гадаад нөлөөгүй бол стандарт техникүүд байдаг.
 - Хэрэв найдвартай байдал, уншигдахуйц, зөв байдал нь үр ашгаас чухал бол функционал програмчлал нь илүү уншигдахуйц ПХ-ийг бий болгоно
 - Зөвийг тогтооход хялбар учир илүү найдвартай байдаг.

ДҮГНЭЛТ >> An Assessment

- Цэвэр бус FL-ний хувьд ч, мөн FL-ний оролцоо бүхий ердийн императив хэлний хувьд ч ПХ-ийн төлөвгүй чухал бүрдэл хэсгийг тусгаарлаж болно.
 - Ингэснээр гадаад нөлөөгүй үргэлж ижил ажиллахыг нь баталж болно.
 - Тиймээс high-order функц нь орчин үеийн PL, парадигмын гол болсон.
- Зэрэгцээ программчлалын механизмууд (for instance, futures or callbacks) нь ихэнхдээ функц дээр суурилдаг.
 - Төлөвийн өөрчлөлтийн оронд утга өөрчлөлтийн ("term rewriting") зэрэгцээ системийг зохиож, хэрэгжүүлэх нь эдгээр системийн нон-детерминизмийг шийдвэрлэхэд хялбар болгодог.
- FL нь аливаа парадигмын PL-ийг зохиоход асар их нөлөөлсөн.
 - FL-ний олон ойлголт, туршилтын үзүүлэлтүүд бусад парадигм руу шилжсэн.
 - Төрлийн систем, генерикс, полиморфизм, төрлийн аюулгүй байдал зэрэг нь бүгд функционал хэлнээс гаралтай (учир нь гадаад нөлөөгүй орчинд тэдгээрийг судлах, хэрэгжүүлэхэд хялбар байдаг).



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА